

Product Requirements Document (PRD)

Project: Member Dashboard for a Club/Gym with Charts and Real-Time Membership Insights

Objective

Build a responsive member dashboard where users can:

- Navigate between sections using tabbed navigation
- View membership information using status cards
- Visualize activity and statistics using Chart.js charts
- Toggle between different real-time datasets without page reloads

The project helps learners practice:

- Component-based UI development
- State management
- Dynamic rendering
- Chart integration using Chart.js
- Handling simulated real-time updates on the frontend

This project is focused only on frontend implementation. Learners can use mock JSON data or simulated APIs.

Background

Modern dashboards are used in almost every product today such as fitness apps, banking apps, learning portals, OTT platforms, and admin panels.

Users expect information to update instantly and interfaces to feel interactive without refreshing the page repeatedly.

This project simulates a real-world membership portal where users can:

- Check membership status
- Monitor usage analytics

- Switch between sections quickly
- View changing data in charts
- Interact with a modern dashboard layout

The dashboard should feel smooth, responsive, and visually organized.

Assumptions

- Users already know basic HTML, CSS, and JavaScript
- Learners understand DOM manipulation
- Learners can work with arrays and objects
- Backend functionality is not required
- Real-time updates can be simulated using timers or mock data toggling

Success Metrics / KPIs

The project will be considered successful if:

- Users can switch tabs without page reload
- Charts update dynamically when data changes
- Membership cards reflect updated states correctly
- Layout works properly on desktop and mobile screens
- Dashboard interactions feel smooth and responsive

Features Overview

The dashboard must contain the following major sections:

Feature	Description
Tabbed Navigation	Switch between dashboard sections
Membership Cards	Display user plan and status
Dynamic Charts	Visualize activity using Chart.js
Real-Time Toggle	Simulate changing datasets
Responsive Layout	Support mobile and desktop screens

User Personas

Persona 1 — Existing Member

A logged-in member who wants to:

- Check current membership plan
- Monitor activity statistics
- Track usage trends
- Quickly switch between sections

Persona 2 — New User

A user exploring the dashboard UI who wants to:

- Understand available features
- Read membership details clearly
- Interact with charts and tabs easily

User Stories

ID	User Story	Priority
US-01	User should be able to switch between tabs	Must Have
US-02	User should see active membership details	Must Have
US-03	User should see charts update dynamically	Must Have
US-04	User should toggle between datasets	Must Have
US-05	User should see responsive layouts on smaller screens	Must Have

ID	User Story	Priority
US-06	User should see animated transitions while switching content	Good to Have

Functional Requirements

Requirement ID	Requirement	Acceptance Criteria	Priority
FR-01	Create tabbed navigation	Clicking tabs changes visible section	Must Have
FR-02	Add membership status cards	Cards display plan name, renewal date, and status	Must Have
FR-03	Integrate Chart.js	At least 2 charts should render properly	Must Have
FR-04	Support dynamic chart updates	Charts update when toggle buttons are clicked	Must Have
FR-05	Simulate real-time updates	Data refreshes periodically using timers/mock updates	Must Have
FR-06	Add responsive styling	Dashboard adjusts for tablet and mobile layouts	Must Have
FR-07	Add loading states	Loading skeletons or placeholders appear before updates	Good to Have
FR-08	Add dark/light theme toggle	User can switch themes	Can Be Done

Dashboard Sections

1. Overview Tab

Should display:

- Welcome message
- Membership cards
- Summary statistics
- Quick actions

Example cards:

- Active Plan
- Membership Expiry
- Total Sessions
- Rewards Points

2. Analytics Tab

Should contain:

- Line chart for activity trends
- Doughnut or pie chart for membership usage
- Dynamic dataset toggles

Example toggles:

- Weekly Data
- Monthly Data
- Yearly Data

3. Settings Tab

Should contain:

- Notification preferences
- Profile settings
- Theme toggle

- Logout button UI

No backend logic required.

Real-Time Data Simulation

Learners should simulate real-time behavior using:

- `setInterval()`
- Mock JSON data
- Randomized values
- State updates

Example scenarios:

- Membership points increasing
- Usage chart refreshing
- Status changing from "Active" to "Expiring Soon"

Suggested Tech Stack

Area	Suggested Technology
Styling	CSS / Tailwind CSS
Charts	Chart.js
Icons	Lucide / Font Awesome
Data Handling	JavaScript Objects & Arrays
State Management	Vanilla JS

UI/UX Guidelines

The interface should:

- Use clean spacing
- Use card-based layouts
- Maintain readable typography
- Use consistent colors
- Avoid cluttered dashboards
- Support hover states and transitions

Suggested layout:

- Sidebar or top tab navigation
- Main content area
- Grid-based cards
- Responsive chart containers

Non-Functional Requirements

Requirement	Details
Performance	Dashboard interactions should feel instant
Responsiveness	Works properly across screen sizes
Accessibility	Proper labels and readable contrast
Maintainability	Organized folder structure and reusable components
Scalability	Easy to add more tabs and charts later

Error State Handling

The UI should handle:

- Empty datasets
- Missing chart data
- Invalid membership status
- Chart rendering failures

Example:

Instead of breaking the UI, show:

"No analytics data available"

Folder Structure Suggestion

```
None
project/
|
├─ index.html
├─ css/
|   └─ styles.css
├─ js/
|   ├── app.js
|   ├── charts.js
|   ├── tabs.js
|   └─ data.js
├─ assets/
|   └─ icons/
```

Future Scope

Possible future improvements:

- Backend integration
 - Authentication
 - Real APIs
 - Notifications system
 - Downloadable reports
 - Live websocket updates
 - User profile editing
 - Multi-user dashboards
-

Non-Goals

The following are out of scope:

- Backend development
- Database integration
- Authentication system
- Payment integration
- Server-side rendering

Learners should focus only on frontend behavior and UI interactions.

Deliverables

Learners should submit:

- Complete frontend source code
- Responsive implementation
- Working charts
- Functional tab system
- Real-time data simulation

Optional:

- Deployment link
- README documentation
- Screenshots or screen recording